

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE CIENCIAS UNAM



Modelado y Programacion

Reporte de Proyecto

Reporte: Base de datos musical

Profesor: Canek Peláez Valdés

Alumno: Soto Mortera David 322301399

CIUDAD DE MEXICO, MAY 2026



Indice

1. Abstract	2
2. Introduccion	3
2.1. Minero	3
2.2. DAO	3
2.3. Interfaz grafica	3
3. Lenguaje (Rust)	3
3.1. Por que lo elegí	3
3.2. Historia	3
3.3. Bibliotecas usadas	4
4. Análisis y diseño	5
4.1. Análisis	5
4.2. Diseño	5
5. Diagrama de clases	8
6. Conclusiones	9
7. Future work	9

1. Abstract

Proyecto 2 de la materia "Modela y Programacion". Se necesito utilizar SQL, y la apropiada forma de manejar esa informacion (DAO). Tambien estuvo presente el uso de una interfaz grafica y el como programarla para adecuarla a las necesidades que se nos presentaban. Y por ultimo la mineria de los metadatos de los .mp3 que era con lo que se poblaria la base de datos y se le mostraria al usuario.



2. Introduccion

2.1. Minero

Se necesitaba algo que fuera recursivo con cada carpeta que encontrara y apartir de ahi buscar los archivos .mp3 y encontrar los metadatos que contienen. Solo especificamente .mp3 por lo que la biblioteca id3v4 fue necesaria y suficiente para este trabajo. Y ocupe la biblioteca de walkdir para pasar recursivamente por las carpetas. (Honestamente no tengo ni idea de que orden toma para ir recursivamente pero funciona.)

2.2. DAO

Para facilitarnos la vida Canek en toda su sabiduria nos dijo que usaramos sqllite dado que no necesitaríamos un servidor de base de datos, que podriamos alojar la base de datos localmente. Con esto tambien nos dijo que mantuvieras la limpieza y estructura de esta misma por lo que necesitaríamos un DAO (Data Access Object). Para que solo un ".objeto"(no tengo objetos en rust lol) interactue directamente con la base de datos y no esten comandos de SQL regados por cada lado.

2.3. Interfaz grafica

Segun Canek esto va a estar mal por que tenemos mal gusto pero bueno. Esto es como se le presenta la informacion que minamos al usuario, todo lo que el minero extrajo y que el DAO guardo. Hay que presentarlo "bonito", entonces no podemos presentar directamente la terminal, por que los usuarios se espantan y se enojan por que son unos inutiles, por lo que necesitamos una forma de presentar la informacion, escogi GTK, por que, no lo se.

3. Lenguaje (Rust)

3.1. Por que lo elegí

Lo use el proyecto pasado y me senti mas comodo esta vez.

3.2. Historia

Checar el proyecto pasado para la historia (no voy a gastar paginas en esto otra vez.)



3.3. Bibliotecas usadas

- **chrono**

^[2]Si no tiene fecha de creacion cual uso?. Por suerte Canek dijo que podriamos usar la fecha de creacion del archivo, pero pedirla directamente algo muy largo raro. Chrono entra a la accion, vuelve eso en un coso al cual le puedo le pedir la fecha de creacion como year y ya.

- **gstreamer**

Para poder reproducir las canciones que estan en la base de datos. Le pasamos la url y la reproduce.^[4]

- **gtk4**

La biblioteca para poder hacer la interfaz grafica con GTK.^[5]

- **id3**

La biblioteca utilizada para extraer los metadatos de UNA unica cancion^[6]. Por eso es necesario que otra biblioteca nos apoye con la recursion.

- **lalrpop**

La usamos para el parser. Construir las expresionas por las cual vamos a buscar en la db. Lo bueno es que lo hace por su cuenta, solo hay que definirlo bien en un .lalrpop^[7].

- **pest**

Igual que lalrpop, esta la usamos para definir los tokens que vamos a aceptar en la busqueda y como vamos construir las expresiones que le vamos al parser. Igual se construye solo, solo hay que definirlo bien en un .pest^[3]

- **rusqlite**

Es la biblioteca necesaria para realizar lo que Canek pidio. Que tengamos la base directa y no en un servidor, aparte de ser bastante rapido y efectivo es medio facil de entender y usar^[8].

- **walkdir**

Su uso fue apoyar al id3, ya que solo puede con un archivo a la vez, por lo que waldir nos ayuda haciendo recursion a travez de toda carpeta que se encuentre dada una ruta especifica. Y justo nos permite manejar entrada a entrada por lo que es facil filtrar cual si es .mp3 y cual no para poder asegurar que no le estamos mandando algo que potencialmente puede explotar al id3^[1].



4. Análisis y diseño

4.1. Análisis

Desde el principio supe como tener que separar las cosas gracias a que Canek fue mas específico y que planee mejor antes de empezar a programar para hacerme la vida mas facil. Sabia que tenia que dividirlo en 4 partes, un minero que sacara los metadatos de la cancion. El DAO que guardara toda la informacion en la db y que hiciera las consultas que se le pidieran. La interfaz grafica para presentarle al usuario la informacion y que pusiera las busquedas que deasea realizar. Y por ultimo el compilador, el que dada una serie de magia negra le dice al DAO que tiene que buscar y por que parametros buscar.

4.2. Diseño

■ Minero

● Audio

Checo que metadatos tiene cada .mp3 y de ahi me aseguro de que si no tienen una etiqueta especifica la agrego por encima. No edito los metadatos, solo agrego lo que yo necesito para guardarlos en structs.

```
1 pub struct Cancion {  
2     pub title: String,  
3     pub artist: String,  
4     pub album: String,  
5     pub genre: String,  
6     pub track: u32,  
7     pub year: i32,  
8     pub path: String,  
9     pub album_path: String,  
10 }
```

● Minero

Paso el walkdir para que sea recursivo y checo cada entrada que tiene para asegurarme que solo le paso archivos con terminacion .mp3 al audio para que no se rompa, tambien me aseguro que sea un archivo, por alguna razon tambien devuelve la carpeta que mina.



```
1 fn es_mp3(entry: &DirEntry) -> bool {  
2     entry  
3         .file_name()  
4         .to_str()  
5         .map(|x| x.ends_with(".mp3"))  
6         .unwrap_or(false)  
7 }
```

■ DAO

El DAO es el unico lugar donde puede haber SQL, es parte de su forma de ser. Tambien es quien lidia con todos los structs de canciones que le pasamos cuando los queremos agregar a la base de datos, lidia con repetidos y con las busquedas que hace el compilador.

Un ejemplo de como lidia con canciones duplicadas es el path, el path es unico y si ya existe pues no agregamos esa cancion.

```
1 fn rola_existe(conn: &Connection, path: &str) -> bool {  
2     let mut rola = conn  
3         .prepare("SELECT 1 FROM rolas WHERE path = ?1")  
4         .unwrap();  
5     rola.exists(params![path]).unwrap()  
6 }
```

■ Interfaz Grafica

La interfaz tiene que lidiar con muchisimas cosas, el hecho de desplegar todo lo que se mino, los botones, las disposiciones que le das a las columnas de las tablas para poder presentar la db. La barra de busqueda, El boton de minado y el como seleccionar carpetas en ese mismo boton. Y yo que me quise ver de melomano, en el panel izquierdo esta todo lo de controlar la musica, botones, la barra de progreso, siguiente y anterior cancion e incluso la imagen que viene con el mp3, para verme aun mas mamon. Podria poner ejemplos infinitos de las cosas que hize en la interfaz pero no es tan interesante, solo fue tardado y confusa. Pero un ejemplo muy bueno es de como abro el explorador de archivos nativos para seleccionar la carpeta.

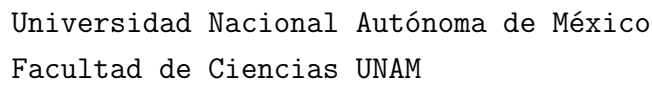


```
1  let browse = entrada.clone();
2  let ref_dialogo = dialogo.clone();
3  btn_browse.connect_clicked(move |_| {
4      let chooser = gtk4::FileChooserNative::new(
5          Some("Seleccionar carpeta"),
6          Some(&ref_dialogo),
7          gtk4::FileChooserAction::SelectFolder,
8          Some("Seleccionar"),
9          Some("Cancelar"),
10     );
11     let entry = browse.clone();
12     chooser.connect_response(move |chooser, response| {
13         if response == gtk4::ResponseType::Accept {
14             if let Some(file) = chooser.file() {
15                 if let Some(path) = file.path() {
16                     entry.set_text(&path.to_string_lossy());
17                 }
18             }
19         }
20     });
21     chooser.show();
22 });
```

■ Compiladorcito

Este fue lo que mas me desanimaba del proyecto, parecia muy complicado y me queria rendir con eso, por eso lo deje hasta el final. Pero resulta y resalta que con las bibliotecas correctas es un trabajo bastante sencillo, solo hay que definir bien las especificaciones del parser y del lexer para que hagan bien su trabajo y definir bien la gramatica que necesitas. A partir de eso ya tienes lo mas dificil y puedes pasarlo al dao para que haga las busquedas.

```
1 token = {
2     and_op
3 | or_op
4 | not_op
```

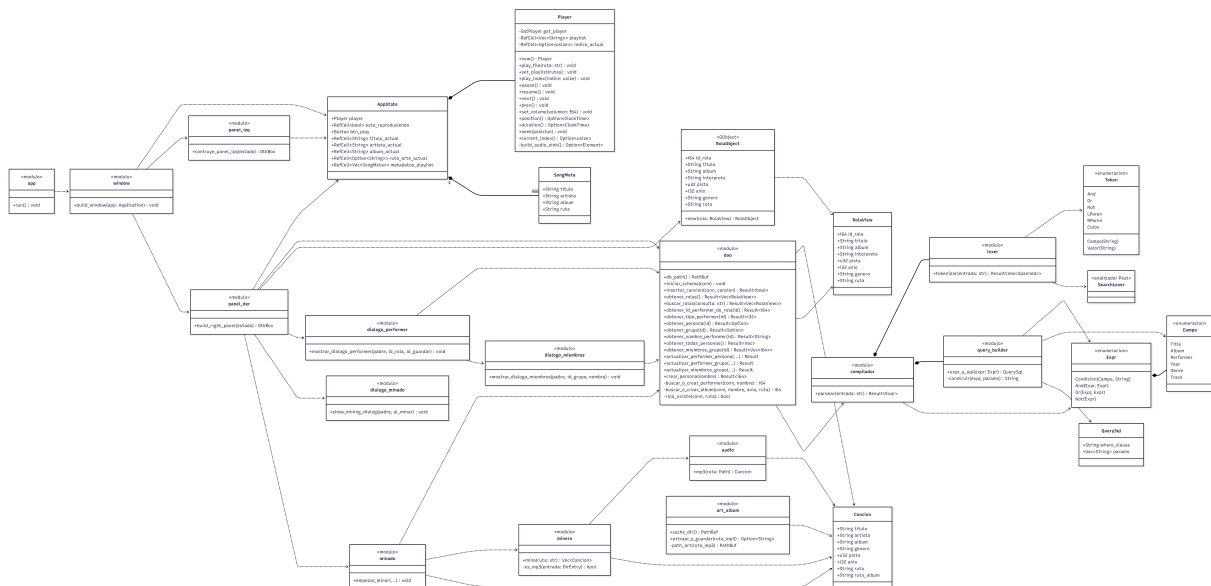


```

5 | lparen
6 | rparen
7 | colon
8 | campo
9 | valor_comillas
10 | valor_simple
11 }

```

5. Diagrama de clases





6. Conclusiones

Este proyecto me enseñó que aun no se manejar mi tiempo, es la cosa en la que mas sufro y pague caro con eso, pero pues que se le va hacer mas que aprender de nuestros errores, me encanto hacer una aplicacion que tuviera que ver con musica, no pude presumir mucho mi catalogo pero me dio una excusa para arreglar los metadatos de mis canciones, aparte de eso me mostro que existen un monton de tecnologias que hacen el trabajo pesado por ti, solo es saber aprovecharlas correctamente y saber que usar. Se que SQL me va a servir en un futuro, fue divertido aprenderlo, aunque fuera por poco tiempo.

7. Future work

Que cambiaría y que podría mejorar?

Mi manejo del tiempo, dejar de posponer las cosas. Me gusto lo del compilador, termino siendo algo muy limpio gracias a las bibliotecas pero definitivamente pedir ayuda antes para saber que chingas hacer desde mucho antes y no desanimarme con un reto que parecia imposible. Tambien no tener que hacer toda la interfaz a mano, Canek menciona Cambalache, si en un futuro tengo que hacer otra interfaz para que sea mas sencillo y menos tedioso. Por ultimo el como manejo el minero, entrego una lista de canciones, que para un biblioteca de canciones no tan grade no es un problema pero para muchisimas puede llegar a afectar el rendimiento de mi aplicacion, deberia separarlo para que regrese de una en una y puede que incluso eso haga que pueda actualizar mi GUI mas rapido.



Referencias

- [1] Rust Documentation. walkdir. <https://docs.rs/walkdir/latest/walkdir/>, 2025.
- [2] Rust Documentation. chrono. <https://docs.rs/chrono/latest/chrono/>, 2026.
- [3] Rust Documentation. The elegant parser. <https://docs.rs/pest/latest/pest/>, 2026.
- [4] Rust Documentation. gstreamer. <https://docs.rs/crate/gstreamer/latest>, 2026.
- [5] Rust Documentation. gtk4. <https://docs.rs/gtk4/latest/gtk4/>, 2026.
- [6] Rust Documentation. id3. <https://docs.rs/id3/latest/id3/>, 2026.
- [7] Rust Documentation. lalrpop. <https://docs.rs/lalrpop/latest/lalrpop/>, 2026.
- [8] Rust Documentation. Rusqlite. <https://docs.rs/rusqlite/latest/rusqlite/>, 2026.